TAL-220 Implementar un pipeline de Integración Continua (CI) con GitHub Actions y análisis estático con SonarQube

Descripción Jira

Objetivo: Implementar un pipeline de Integración Continua (CI) con GitHub Actions y análisis estático con SonarQube para validar automáticamente la calidad y seguridad del código en cada Pull Request.

Descripción: Configurar un flujo de trabajo automatizado (workflow) en GitHub Actions que intercepte los Pull Requests hacia las ramas protegidas (dev y main). El pipeline se encargará de instalar las dependencias, ejecutar las suites de pruebas unitarias existentes e integrar el código con SonarQube. SonarQube realizará un análisis estático para detectar vulnerabilidades (Security Hotspots), code smells y medir la cobertura, actuando como una barrera de calidad estricta (Quality Gate) que asegure la estabilidad del sistema antes del despliegue.

Criterios de Aceptación:

1. El workflow de GitHub Actions se dispara automáticamente al abrir, actualizar o sincronizar un Pull Request hacia dev o main.
2. El botón de "Merge" en GitHub queda bloqueado y reporta error si falla la ejecución de los tests unitarios de la aplicación.
3. El botón de "Merge" en GitHub queda bloqueado si SonarQube detecta que no se superan las reglas establecidas en el Quality Gate (ej. vulnerabilidades críticas o bugs nuevos).

Dependencias: Requiere la existencia de pruebas unitarias base (ej. TAL-181).

Notas

GitHub Actions

Motor de automatización (CI/CD) que se integra en los repositorios. Funciona a través de una *YAML* que le dicta una serie de pasos que el servidor de GitHub debe ejecutar de forma automática cada vez que alguien abre una *Pull Request* hacia *dev*.

En InfraManager:

- **Revisión de quiebres de compilación:** GH Actions levanta un entorno temporal e intenta hacer el build del proyecto. Si el código de la PR rompe la compilación, marca la PR con

una X roja.

- **Pruebas automáticas:** Ejecuta todos los tests que se tengan. Si un test falla, el PR se bloquea.
  - Jest para React/NestJS, Pytest para Django.
- **Estilo y formato (Linters):** Ejecuta ESLint o Prettier. Si el código está mal identado o tiene variables sin usar, el sistema lo rechaza automáticamente.

SonarQube

Mientras GitHub Actions ejecuta comandos, SonarQube lee y analiza lógicamente el código escrito.

- Se integra directamente en el flujo de GitHub Actions.
- **Detección de Code Smells:** Avisa de funciones demasiado largas, código duplicado, complejidad ciclomática excesiva.
- **Bugs y Vulnerabilidades:** Analiza fugas de memoria, variables que podrían ser nulas y causar un crash, o brechas de seguridad (inyecciones SQL, tokens expuestos).
- **Mide el Coverage:** Indica qué porcentaje del código está cubierto por pruebas.
- **Quality Gate:** Se puede establecer la regla *No se aprueba ningún PR que tenga más de 0 bugs críticos o menos del 80% de cobertura de pruebas.*

Integración con Kubero

Sobre Kubero

Concretamente, lo que tenemos es que usar es «una plataforma CI/CD educativa para la UCT basada en Kubero, un PaaS de código abierto alternativo a Kubernetes. Los estudiantes pueden publicar sus aplicaciones con HTTPS y redespliegue automático desde GitHub Actions».

- La documentación de esta plataforma basada en Kubero tiene documentación en español y su propio fork donde se corrigen errores de seguridad y funcionalidad.

GitHub Actions y Sonarqube son el **CI**: su trabajo es **integrar** el código nuevo y asegurarse de que sirva, compila y sea seguro.

Kubero es el **CD**: Su trabajo es tomar el código de la rama principal, meterlo en contenedores Docker y ponerlo a funcionar en internet.

- Si se usa Kubero sin GHA, un problema en la rama principal puede romper la compilación en tiempo real y hacer caer la plataforma en producción. GHA hace lo mismo que Kubero, levanta una VM, hace los tests y compila el código.
- **Se imposibilita el unir a la rama main un código que romperá el sistema.**

Proceso de desarrollo de la tarea

GitHub Actions y SonarQube a la vez

1. Lo primero es crear `.github/workflows/ci.yml` para PR hacia las ramas dev y main, con los comandos correctos de PNPM para cada microservicio para ejecutar correctamente las pruebas.
2. Se prueba el funcionamiento de los tests en el repo local:

```
    for servicio in ms-auth ms-activos ms-incidencias ms-notificaciones; do
  echo "===== $servicio ====="
  (cd "$servicio" && pnpm test -- --runInBand)
done

echo "===== frontend-web ====="
(cd frontend-web && pnpm test:unit)
```

```
for servicio in ms-activos ms-incidencias; do
  echo "===== $servicio ====="
  (cd "$servicio" && pnpm test)
done
```

3. El exámen fue el siguiente, por lo que se arreglan esos specs, los faltantes se integrarán en otra TAL si es que da tiempo:

Microservicio	Suites	Pruebas	Resultado
`ms-activos`	4/4	32/32	✓ Aprobado
`ms-incidencias`	14/14	107/107	✓ Aprobado
`ms-auth`	5/7	28/32	✗ 4 fallidas
`ms-notificaciones`	—	0	Sin pruebas detectadas
`frontend-web`	—	—	Sin script `test:unit`

4. Una vez resuelto eso, el CI YAML debe configurarse para ejecutarse cad vez que se abra o actualice un PR.

Configuración yaml	función
`on: pull_request`	Activar pipeline.

Configuración yaml	función
`permissions: contents: read`	Limita los permisos del workflow a la lectura del repositorio.
`jobs: tests`	Ejecuta los tests de los tres MS.
`matrix.service`	Evita la repetición de configuración de pruebas para cada MS.
Comandos `pnpm` de instalación y testing	
`needs: tests`	Impide ejecutar SonarQube si no se aprobaron los tests.
`sonarqube-scan-action`	Analiza el código en busca de problemas de calidad y seguridad.
`sonar.qualitygate.wait=true`	Espera el resultado del QualityGate y falla si no se aprueba

5. Después de esta configuración YAML, se requiere conectar el repo de GitHub con SonarQube y obtener los datos que necesita `ci.yml` para ejecutar el análisis. *SonarQube Cloud nos permite no levantar un servidor de análisis local propio!*
    - Las variables `SONAR_ORGANIZATION` y `SONAR_PROJECT_KEY` deben ser configuradas en el repositorio de GitHub (GH Actions) y el secret `SONAR_TOKEN` debe ser agregado también (solo lo podrá ver quien lo agregó).

El análisis estático, la cobertura y el Quality Gate.

6. Con el Workflow/Pipeline ya funcionando, como agregados podemos proteger la ramas para exigir checks exitosos antes de permitir mergings.
7. Para probar esta tarea, basta con crear un PR de prueba y verificar que un fallo bloquee el merge.

Commit(s)

- Arregla `auth.service.spec.ts` y `user.repository.spec.ts`.
- Crea el archivo `ci.yml`, que define el workflow de GitHub Actions para definir y coordinar todo el proceso de CI, con procesos encargados de GitHub Actions propiamente tal y otros encargados de SonarQube pero invocados propiamente tal por GH Actions.